Call by value and call by reference/address:

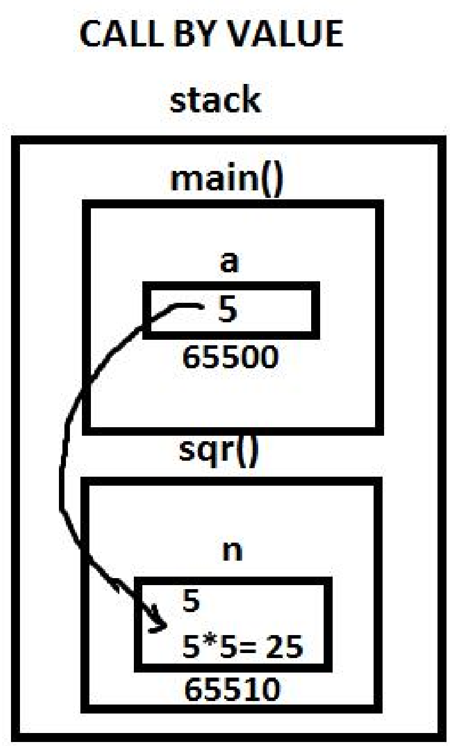
Passing parameters to the functions: [parameter passing techniques]

In C-Language, we can send the arguments to the functions in 2 ways.

1. **Call by value / pass by value.**
2. **Call by address / pass by address. [call by reference]**

Call by value / pass by value:

In call by value we are sending actual parameter value to the formal parameter. Later there is no relation is maintained in between actual and formal parameters. Due to this any change in formal parameter doesn't effects the value of actual parameter.



Eg: 1

```
#include<stdio.h>
#include<conio.h>

void  sqr(int n)

{

n = n * n;

}  /* n  deleted after the function execution
*/

void main()

{
```

```
int a=5;
clrscr();
printf("Before function call a = %d\n" ,a);
sqr(a);  /* fun calling */
printf("After function call a = %d", a):
getch();
}
```

<u>Output:</u>

Before function call a = 5

After   function call a = 5

Eg: 2 swapping of two integers

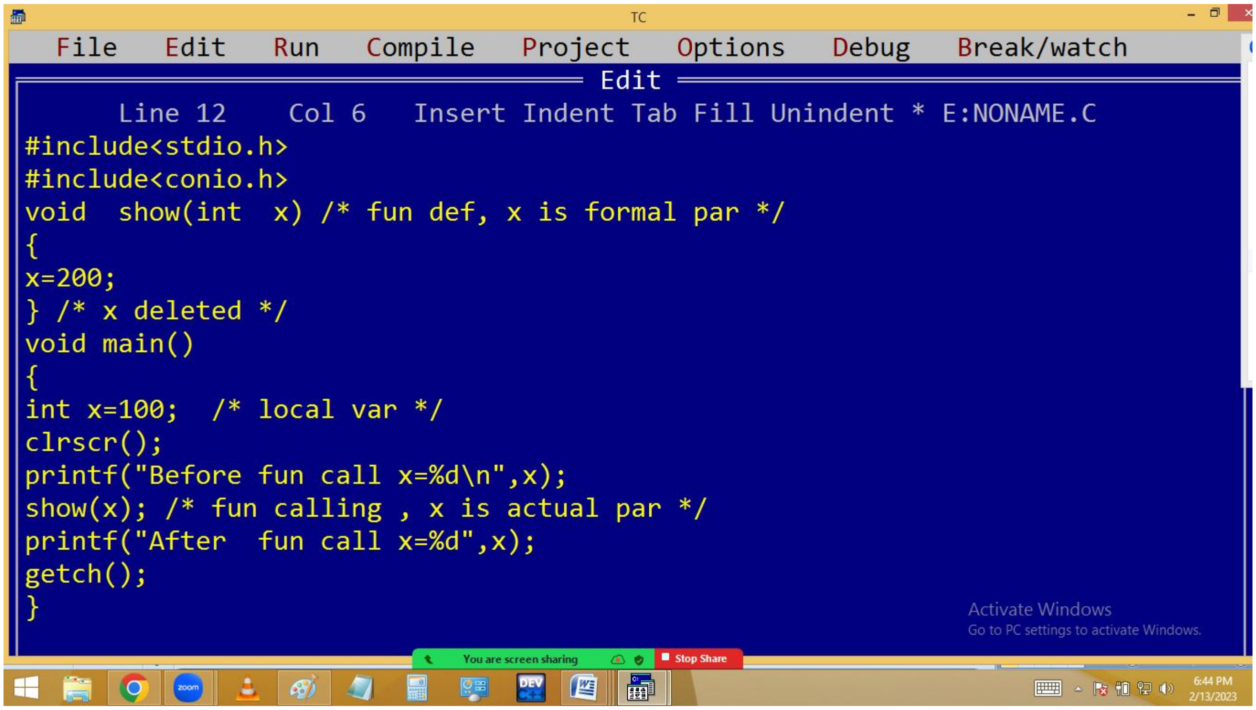
```
#include<stdio.h>
#include<conio.h>
void  swap(int  a, int  b)
{
int  temp=a;
a=b;
b=temp;
}
```

```
void  main()
{
int  a=5, b=7;
clrscr();
printf("Before fun call a=%d, b=%d\n" , a , b);
swap(a, b);
printf("After fun call a=%d, b=%d", a , b);
getch();
}
```

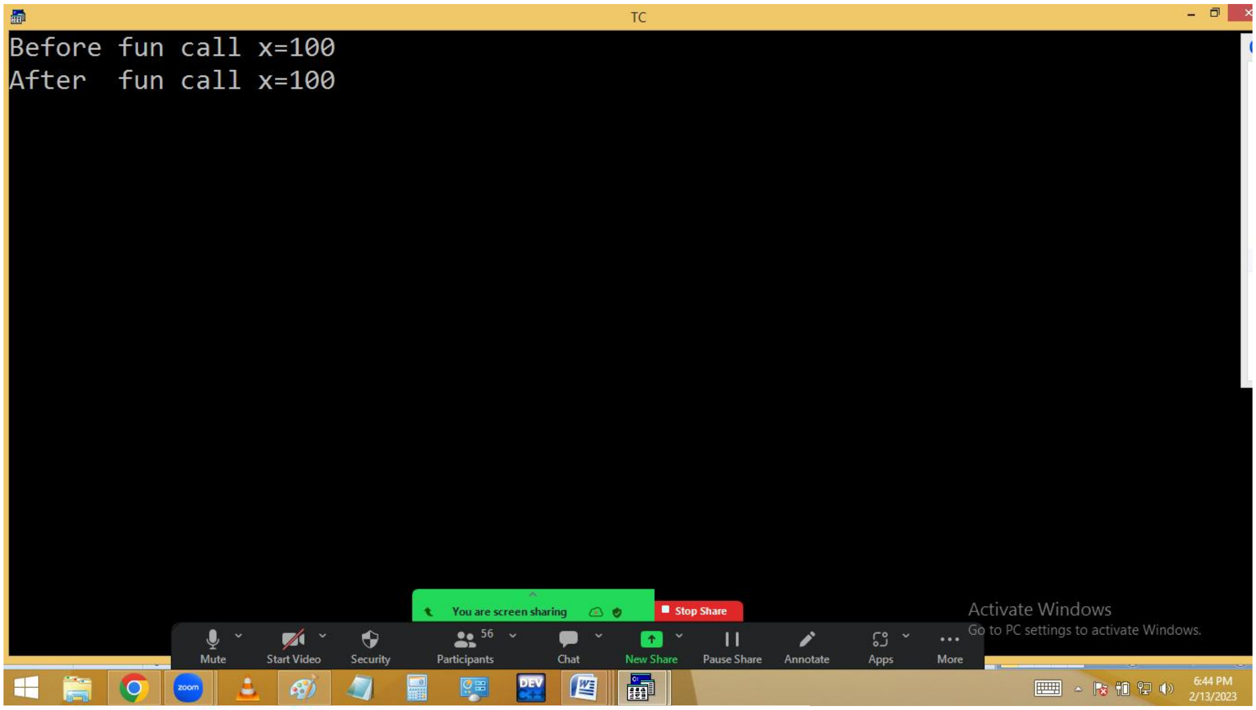
Output:

Before fun call a=5, b=7

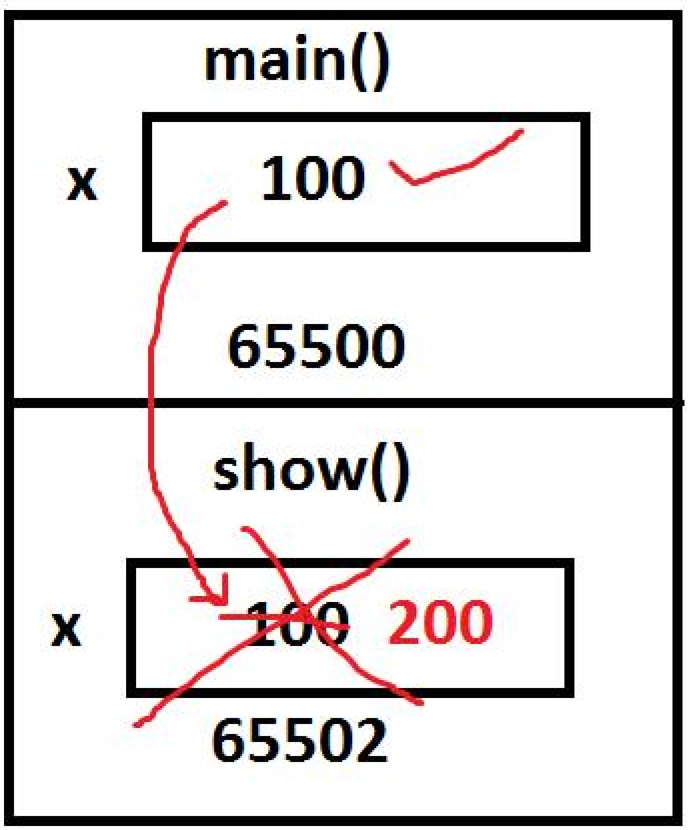
After fun call a=5, b=7




```c
#include<stdio.h>
#include<conio.h>
void  show(int  x) /* fun def, x is formal par */
{
x=200;
} /* x deleted */
void main()
{
int x=100;  /* local var */
clrscr();
printf("Before fun call x=%d\n",x);
show(x); /* fun calling , x is actual par */
printf("After  fun call x=%d",x);
getch();
}
```



Call by /pass By value



Call by address [Reference]:

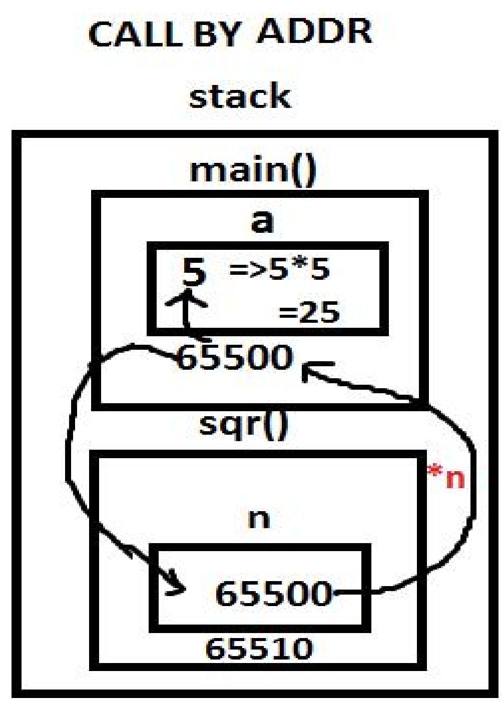
In call by address, the address of actual parameter is passed to formal parameter. Due to this the formal parameter should be declared as a pointer. Then only the formal parameter receives the actual parameter address. Due to this any changes in formal parameter effects in actual parameter address i.e. actual parameter value.

Hence pointers allows the local variables to access outside the functions and this process is called call by address / reference.

It is very much useful in handling the strings, arrays etc outside the functions.

Eg: 1

```
#include<stdio.h>
#include<conio.h>

void  sqr(int *n)

{

*n = *n   *   *n;

}

void main()

{

int a=5;

clrscr();

printf("Before function call a = %d\n " ,
```



```
a);
sqr(&a);  /* fun calling with address */
printf("After function call a = %d " , a);
getch();
}
```

Output:

Before function call a = 5

After   function call a = 25

Eg: 2 Swap of two integers

```
#include<stdio.h>
#include<conio.h>
void  swap(int  *a, int  *b)
{
int  temp=*a;   *a = *b;  *b=temp;
}
void  main()
{
int  a=5, b=7;
clrscr();
```

printf("Before fun call a=%d, b=%d\n", a ,b);

swap(&a, &b);

printf("After fun call a=%d, b=%d", a ,b);

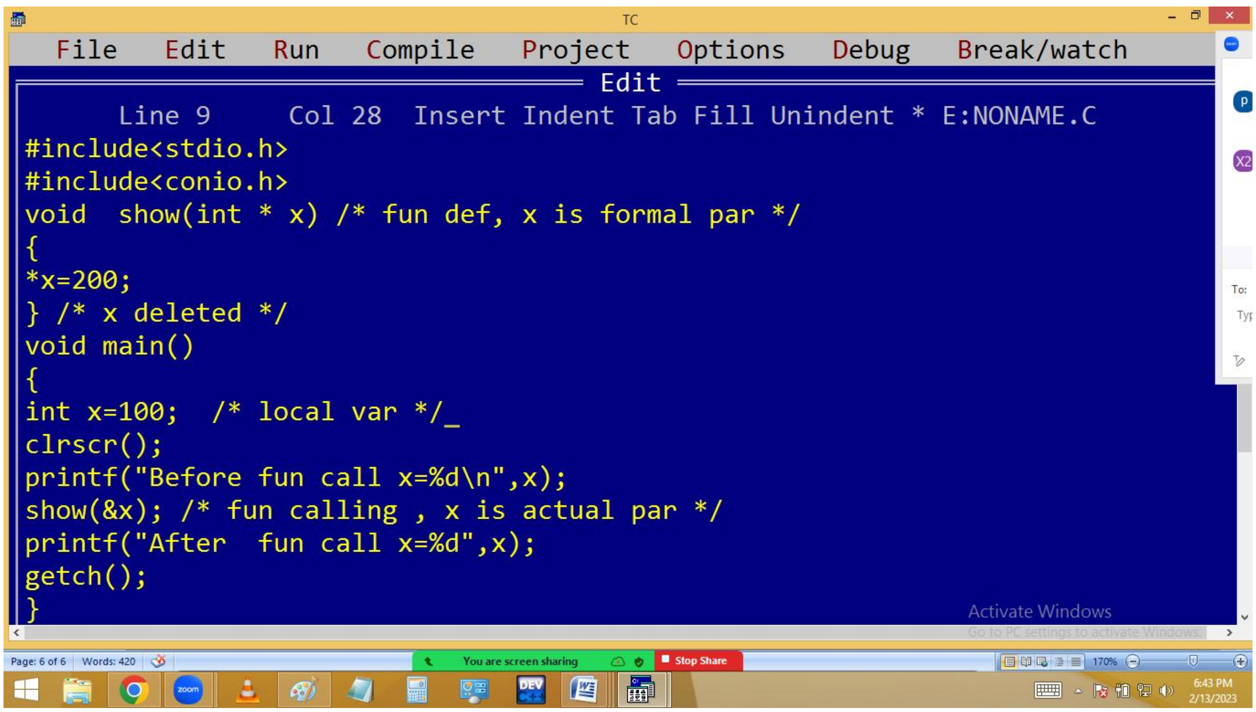
getch();

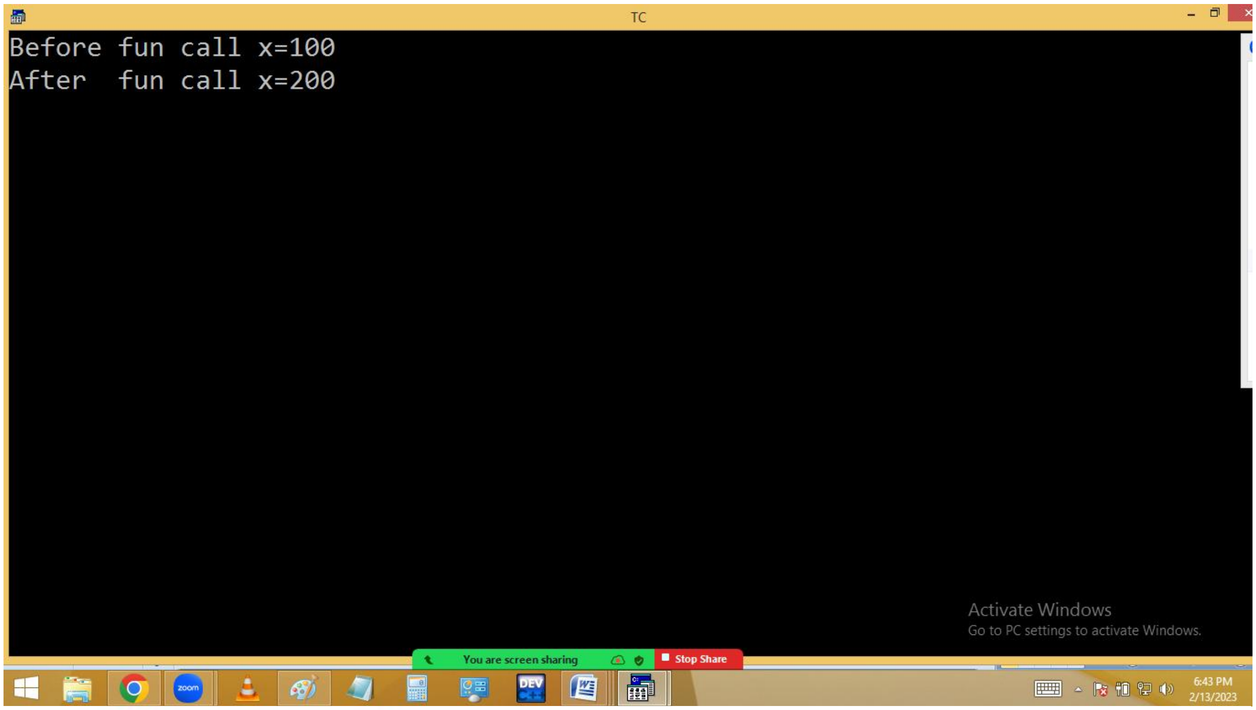
}

Output:

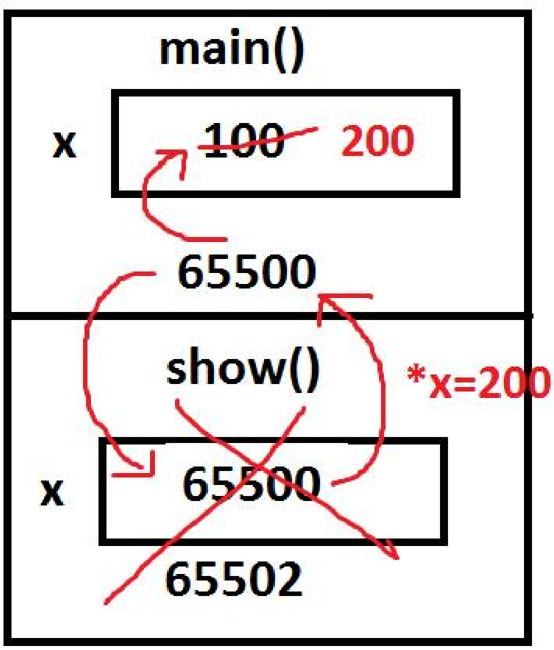
Before function call a=5, b=7

After   function call a=7, b=5





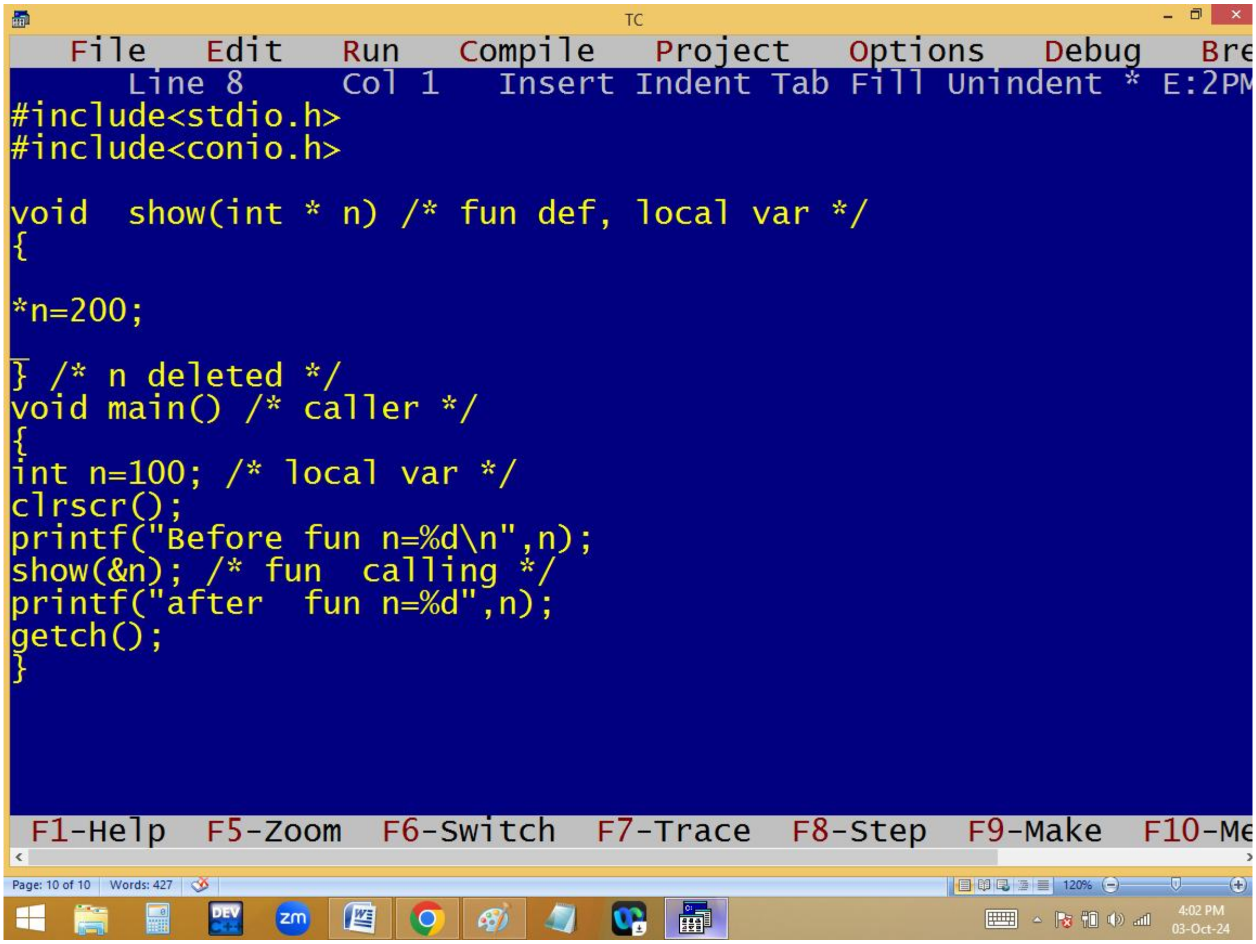
call / pass by address



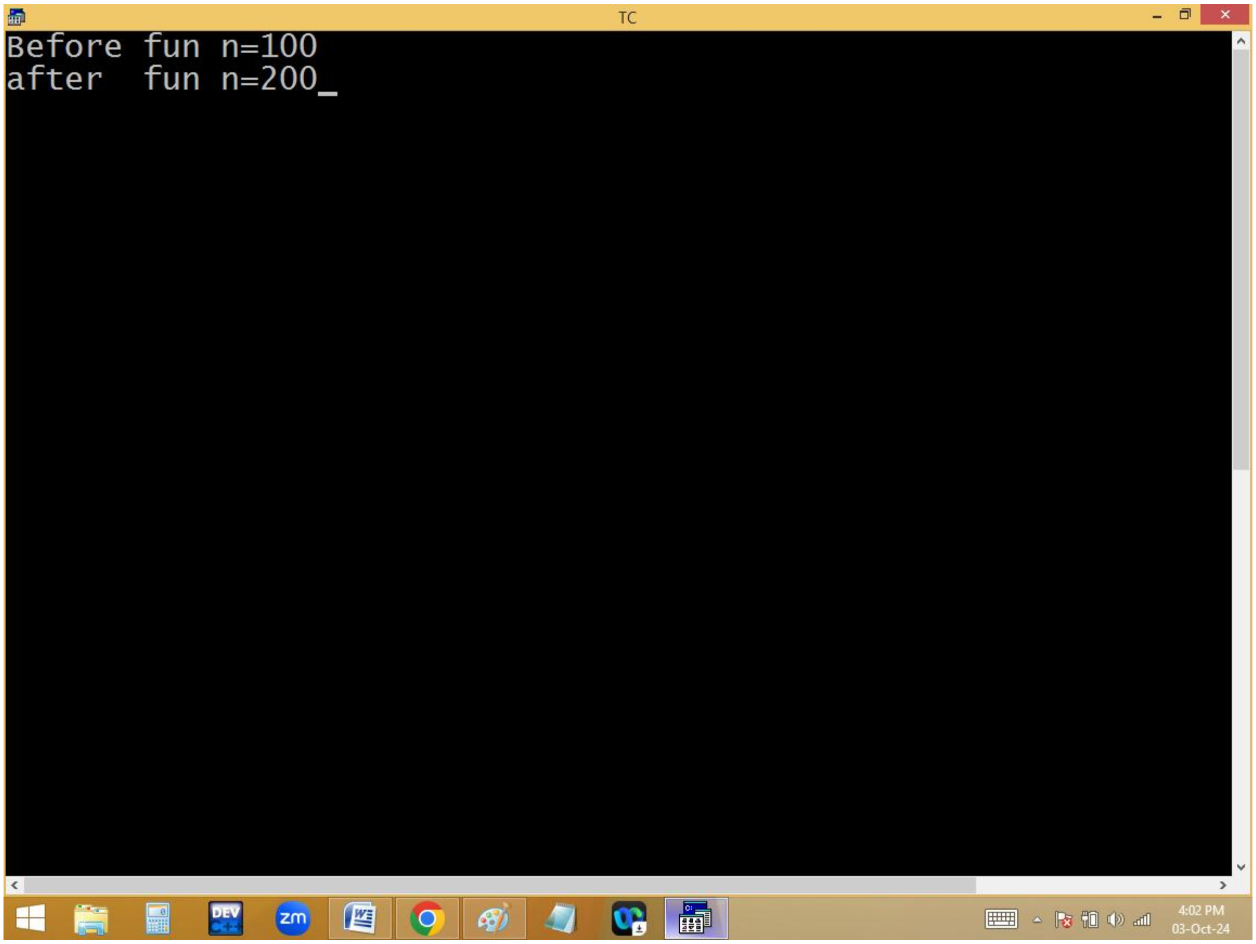
*x = 200

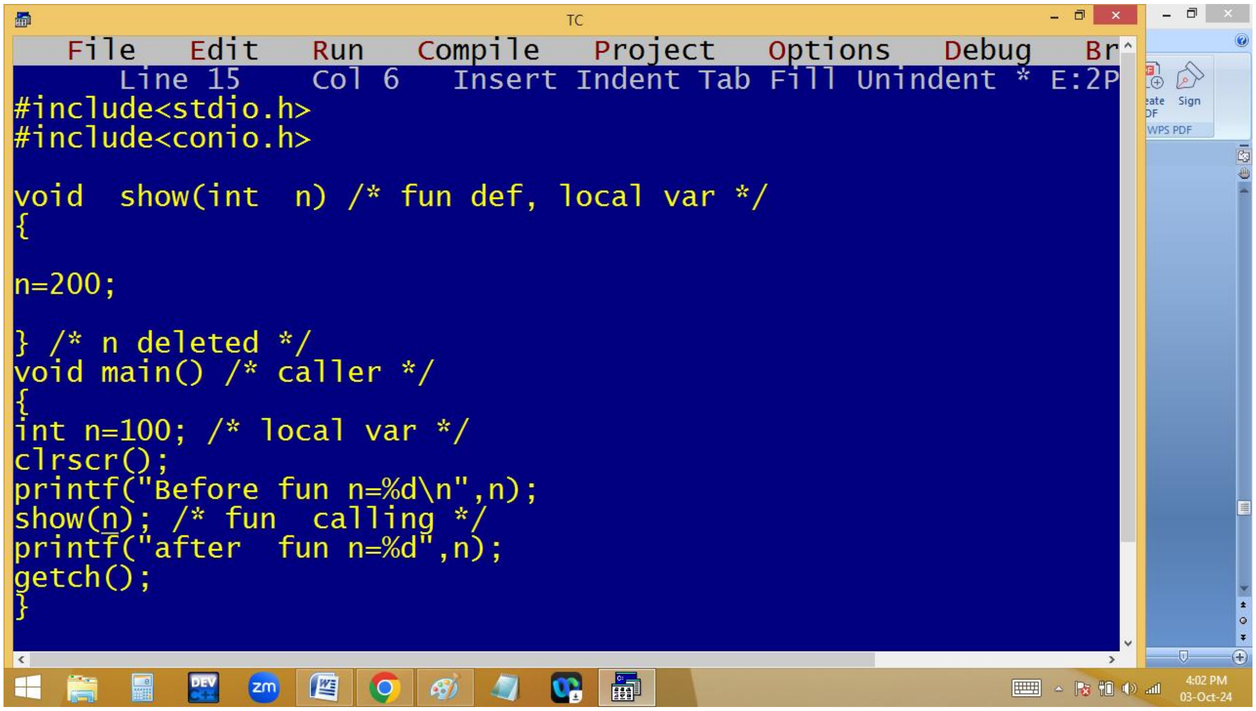
x value is 65500

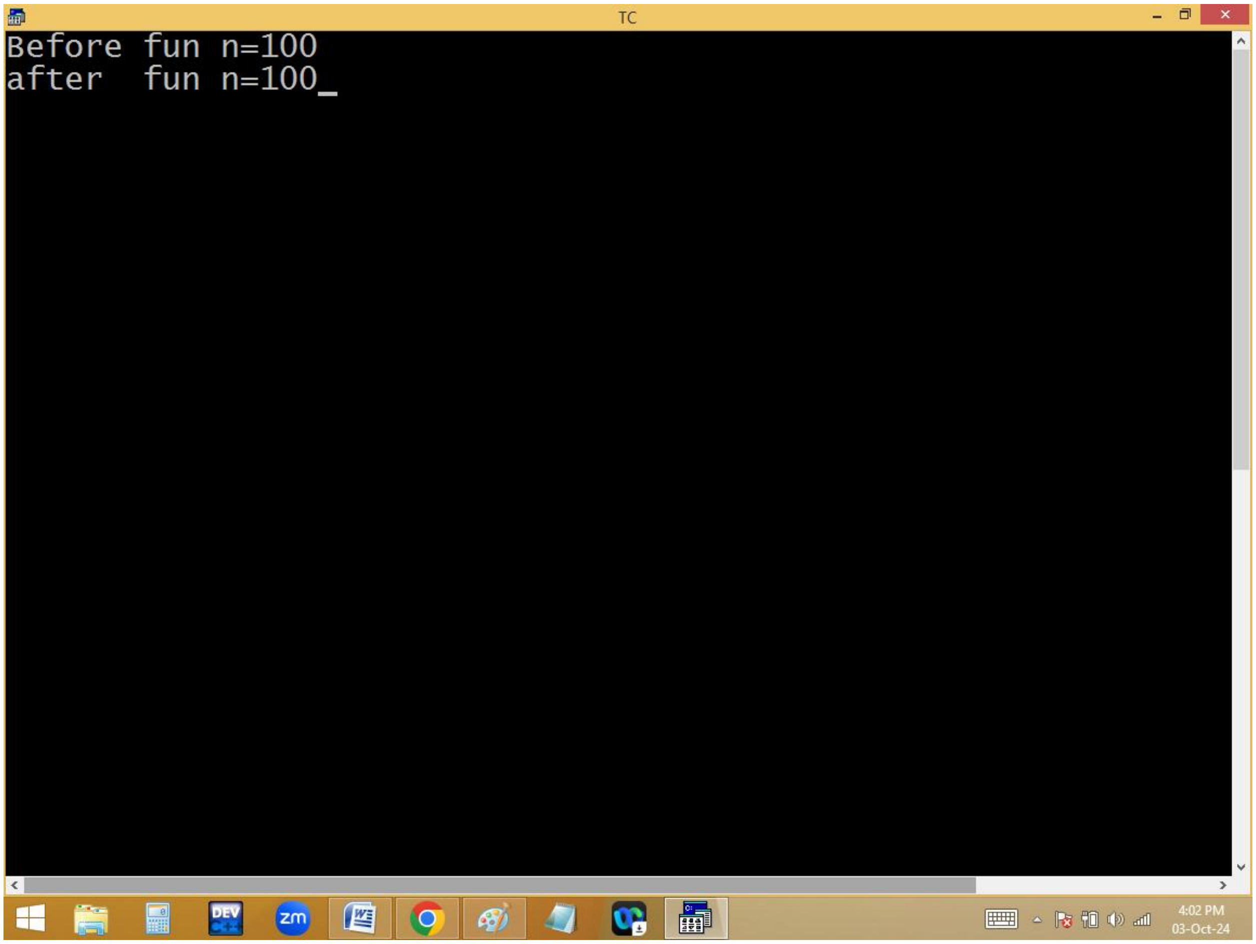
* means value at 65500 = 200




```c
#include<stdio.h>
#include<conio.h>

void  show(int * n) /* fun def, local var */
{

*n=200;

} /* n deleted */
void main() /* caller */
{
int n=100; /* local var */
clrscr();
printf("Before fun n=%d\n",n);
show(&n); /* fun  calling */
printf("after  fun n=%d",n);
getch();
}
```


TC
Before fun n=100
after fun n=200
4:02 PM
03-Oct-24


TC
File Edit Run Compile Project Options Debug Br
Line 15 Col 6 Insert Indent Tab Fill Unindent * E:2P
#include<stdio.h>
#include<conio.h>
void show(int n) /* fun def, local var */
{
n=200;
} /* n deleted */
void main() /* caller */
{
int n=100; /* local var */
clrscr();
printf("Before fun n=%d\n",n);
show(n); /* fun calling */
printf("after fun n=%d",n);
getch();
}
Sign
WPS PDF
4:02 PM
03-Oct-24


TC
Before fun n=100
after fun n=100
4:02 PM
03-Oct-24

text/code area

call by value/ pass by value   call by addr/pass by addr